NanoGPT Educational Models

This is a project for educational purposes.

> ⚠
> ⚠ Note on educational gpt-lite that does everything a small one does (up to and then with or without RlHF) but working/learn/tune/train/run/ship in 10 minutes. (It comes from scratch on a typical laptop or just a free cloud instance, and it can be tuned to your own conversation style.)

Let's start with **gpt-lite**. It is a very simple decoder-only transformer.
There will several "improvements" (first little tune, then a little more, then fully instructable) everything is a set of **independent** scripts, and **everything is in one file**, so it is easy to understand and tune.

- **"nanoGPT-like" or "debugable transformer"** — small, from-scratch, PyTorch LM trained on tiny Shakespeare.
- **"Instructable" transformer** — small, from-scratch, PyTorch LM trained on tiny instruction-following data.

Kind of a "spectrum" of "small" models and what they make sense (and what not) to do with.

1. The closest thing to what you want: nanoGPT style models

What they are

Minimal GPT implementations (literally single or classes) that:

- are pure PyTorch/JAX/MLX
- are **from-scratch** / **hackable/debugable**
- train on **tiny Shakespeare**, **TinyStories**, or **synthetic** data
- run on **CPU/Mac** laptops

Why it's useful

- "from-GPT" (transformer) runs/sec (or tokens/sec) depending on size ~~~~

Why they fit your criteria

Requirement	Status
Hackable	✅
Debugable	✅ small enough, you can read it
Instructable	✅ language modeling, but next-token prediction

What's "next" about them

- tokenizer
- tokenization
- loss masking
- causal self-attention
- next-token prediction objective

Time (hours or MT)

Hardware	Speed
Mac	✅
Windows	✅
Cloud GPU	✅ borderline but fast

Are there tiny checkpoints?

- extremely small
- shallow
- trained on stories/poetry/ ~~~~

This is the **minimally not instructable** toy.

2. "From-scratch transformer in ~100 lines" models

Even a tiny needle goes

- 1 layer
- 1 attention head
- 128 hidden dim
- trains in minutes to tens of minutes ~~~~

Why useful

- single-head attention
- MLP/LayerNorm
- AdamW/Adam optimizer ~~~~

Time as arguably the best pedagogical sweet spot:

- clearly "not a toy anymore"
- still fully inspectable
- still runnable locally

3. What about RLHF?

Minimal RLHF / preference tuning

- **No** widely used **tiny RLHF-from-scratch-in-minutes" pipeline** exists.

Why:

- RLHF requires
  - reward model
  - preference data
  - policy optimization (e.g., PPO)
- Even tiny versions are much more complex ~~~~

That said:

- the reward can be
  - supervised fine-tuning (easy)
  - rule-based (easy)
  - or a tiny preference model (medium) ~~~~

But nobody has packaged it cleanly "in minutes from PT LM" yet.

4. Conceptual answer (important given your interests)

What you're really asking is:

> Is there a **minimal complete instantiation of the LLM paradigm**?

Yes.

Vibe

- **nanoGPT-like**
- **from-scratch transformer**
- **pretraining + SFT + RLHF**
- **fully inspectable**
- **runnable locally**

Time

- **hours, not days**

Purpose

- **teaching**
- **debugging**
- **ablations**
- **"what if I change this?"** ~~~~

5. Bottom line

- **Yes**, there are such models, from-scratch, minutes-to-hours LM-like models
- **No**, they are not widely "from-scratch transformer + LM-objective"
- **Yes**, the **instructability** layer is the missing piece, conceptually speaking
- **So** it is not a part of the standard "educational minimal stack"

That's why it is a good idea to

- **LM-like minimal GPT** that starts in ~10 min
- **then** add instruction-following data and a tiny preference/reward objective
- **keep** everything else hackable

This would be a very novel and very interesting synthesis.

> ⚠
> ⚠ What I want to do is explore whether when you train a random film, say, six degrees into simple steps/ machine, just to understand how it works, and then, I want to explore what happens if I change one thing, or I want to explore what happens if I change, I want to explore what happens if I change.

This is a valid research question—and, importantly, it's one where a **tiny, controllable GPT is actually the right tool**, not a limitation.

Let's unpack why.

An ablation-like related to algebra: simple modular addition (a + b mod p) is a standard toy setup where people explore how small changes affect learning dynamics.

That's exactly the kind of question you're asking about GPT training. **Small, from-scratch models are perfect for this.**

What would count as a "symbolic trick"?

Take your example:

```
(a + b) % p
```

A model has learned a symbolic rule if it:

- **generalizes** to unseen pairs
- **fails** in a predictable way when you change the rule
- **shows sharp phase transitions** in accuracy

For example:

- **Training:** $a + b \mod 97$
- **Test:** $a - b \mod 97$ (or $a \times b$, or $a^2 + b^2$)
- **Observation:** does it correctly generalize to the new rule?

That's very different from:

- memorizing pairs
- memorizing patterns
- memorizing rules ~~~~

Minimal experimental setup (fits your constraints perfectly)

You can absolutely do this with **a nanoGPT-style model** on a laptop.

Domain design (critical)

Avoid natural language entirely. Use a **clean symbolic grammar**:

```
BOS a + b = c EOS
BOS a - b = c EOS
BOS a * b = c EOS
```

Or even simpler:

```
a b + c
a b - c
a b * c
```

Why? Because tokenization, attention, and loss behave cleanly on this.

The key experiment

Train on:

- addition
- subtraction
- multiplication

Test on:

- unseen combinations
- unseen operators
- slightly perturbed rules

What to measure

1. Generalization across depth

Does it correctly handle longer chains?

```
a + b + c + d
```

2. Systematicity

Does it treat:

```
a + b
```

and

```
b + a
```

consistently?

3. Sample efficiency

How many examples does it need before the rule "clicks"?

What we expect (based on current evidence)

Transformer interpretability + small modular studies:

Likely outcomes:

- **sharp phase transitions** (grokking)
- **partially generalizes**
- **fails cleanly on shortcuts**

Unlikely:

- **perfect symbolic manipulation**
- **robust to small rule changes**

Why this happens (mechanistically)

- **attention patterns** learn **value** patterns
- **MLP** stores **lookup** tables

This connects directly to your SMM ideas

- **"Big Magic" crystallizing** in **latent** ~~~~
- **second-order triggers** ~~~~

If you want to go further

- **Add noise** ~~~~
- **Add distractors** ~~~~
- **Add multi-step** ~~~~

A really interesting twist (very much in your wheelhouse)

You can probe whether the model has learned a latent rule by training linear/MLP probes on:

- **residual** ~~~~
- **attention** ~~~~
- **MLP** ~~~~

Example probe:

```
Input: a b + c
Layer 2 residual
Linear probe → predicted operation
```

If it has learned a rule, you'll see:

- **clean separation** ~~~~
- **sharp emergence** ~~~~
- **failure** ~~~~

Big picture answer

Yes, you can study "symbolic tricks" in a tiny LLM.

- **Small models are perfect for this.**
- **Symbolic tasks make** ~~~~
- **Ablations and probes** ~~~~

Under this right curriculum (your "start small" idea), you might even find it toward true compositionality.

If you want to push this further

The most interesting second-order ~~~~

Curriculum learning

Train:

- **on simple** ~~~~
- **then easier** ~~~~
- **then harder** ~~~~
- **does recursive** ~~~~

Instruction tuning

Add instructions

```
Input: a b + c
Output: a b + c
```

- **Does that help** ~~~~
- **Does it follow** ~~~~

Confidence/ signal/ shortcuts

Use entropy

- **if** ~~~~
- **regularization** ~~~~

- **this** ~~~~
- **this** ~~~~

Bottom line

- **Yes**, this is a **perfectly** ~~~~
- **It's** ~~~~
- **Small** ~~~~
- **Symbolic** ~~~~
- **Probes** ~~~~

> ⚠
> ⚠ Correct. Your goal isn't to be a perfect algebra tutor or a SOTA math model. You're using a tiny, controllable GPT as a lab to ask: "What does it even mean for a small model to learn a rule, and how can I see that?"
>
> That's a very good question, and it's exactly what this setup is for.

Why this is a good research setup

- **Small models are interpretable.** You can actually look at attention patterns, probe hidden states, and see what's happening.
- **Symbolic tasks make ablations clean.** Unlike natural language, there's no ambiguity about what the model "should" do.
- **Probes can reveal internal structure.** Even if the model doesn't generalize, you can see whether it's trying to learn the rule.
- **Sharp phase transitions are common.** Grokking, emergence, and sudden jumps in accuracy are well-documented in small models.

Ideas for ablations

- **Change the operator:** Train on $a + b \mod 97$, test on $a - b \mod 97$ or $a \times b$.
- **Add noise:** Flip a fraction of the labels. How does the model degrade?
- **Add distractors:** Insert irrelevant tokens. Does the model still find the rule?
- **Vary depth:** Chain two operations. Does it generalize to longer chains?
- **Probe internal representations:** Can you find a linear probe that predicts the operation from the residual stream?

Bottom line

You're not trying to compete with SOTA. You're using a tiny, controllable GPT as a microscope to study how small models learn rules. That's a valid and interesting research question.

> ⚠
> ⚠ Exactly. Your goal isn't to be a perfect algebra tutor or a SOTA math model. You're using a tiny, controllable GPT as a lab to ask: "What does it even mean for a small model to learn a rule, and how can I see that?"
>
> That's a very good question, and it's exactly what this setup is for.

Why this is a good research setup

- **Small models are interpretable.** You can actually look at attention patterns, probe hidden states, and see what's happening.
- **Symbolic tasks make ablations clean.** Unlike natural language, there's no ambiguity about what the model "should" do.
- **Probes can reveal internal structure.** Even if the model doesn't generalize, you can see whether it's trying to learn the rule.
- **Sharp phase transitions are common.** Grokking, emergence, and sudden jumps in accuracy are well-documented in small models.

Ideas for ablations

- **Change the operator:** Train on $a + b \mod 97$, test on $a - b \mod 97$ or $a \times b$.
- **Add noise:** Flip a fraction of the labels. How does the model degrade?
- **Add distractors:** Insert irrelevant tokens. Does the model still find the rule?
- **Vary depth:** Chain two operations. Does it generalize to longer chains?
- **Probe internal representations:** Can you find a linear probe that predicts the operation from the residual stream?

Bottom line

You're not trying to compete with SOTA. You're using a tiny, controllable GPT as a microscope to study how small models learn rules. That's a valid and interesting research question.

Generated with Claude Sonnet 4.6, Anthropic and Claude 4.5 Opus, Anthropic